SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSIGNMENT REPORT

Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

Course Code & Name	CSA11 - Object Oriented Analysis and Design
Course Outcomes	CO1, CO2, CO3
Bloom's Taxonomy Level	BL3 - Apply, BL4 - Analyse, BL5 - Evaluate, BL6 - Create
SDG Mapping	SDG 3, SDG 9, SDG 11, SDG 13
Assignment Weightage	100%
Student Name	_____
Register Number	_____
Date of Submission	_____

Annexure A - Assignment Code of Conduct

I _____ (Name / Register Number) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____

Table of Contents

1. Problem Analysis and Requirement Summary

Disasters such as floods, earthquakes, industrial accidents, and building collapses create environments that are too dangerous, too large, or too time-critical for human rescue teams to survey unaided. The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is proposed as a software-controlled ecosystem that lets emergency personnel register an incident, define the geographic zones affected by it, and dispatch autonomous drones equipped with cameras and environmental sensors to survey those zones, detect victims and hazards, and relay live data back to a central command system so that rescue teams can be coordinated quickly and safely.

From the given scenario, the system is understood to be a real-time, distributed, safety-critical application in which software objects must represent both administrative concepts (incidents, missions, teams, resources) and physical/autonomous entities (drones, sensors, cameras) that operate with only intermittent human supervision. This dual nature drives the object identification, relationship design, and behavioural modelling carried out in this report.

1.1 Functional Requirements

- FR1: The system shall allow authorized emergency personnel to register a disaster incident with type, severity, and location.
- FR2: The system shall allow coordinators to define one or more affected zones for an incident using geographic coordinates.
- FR3: The system shall authenticate users and enforce role-based access (Coordinator, Operator, Field Team Member).
- FR4: The system shall assign available drones and operators to rescue missions and plan an optimal route to the affected zone.
- FR5: Drones shall autonomously navigate to assigned zones, capture images/video, and collect environmental sensor readings (thermal, gas, etc.).
- FR6: The system shall transmit sensor data and live video feeds from drones to the central command system in real time.
- FR7: The system shall analyse incoming data to detect victims or hazards and automatically generate emergency alerts.
- FR8: The system shall notify and coordinate field rescue teams based on generated alerts.
- FR9: The system shall track and display real-time drone status (location, battery, health) to operators and coordinators.
- FR10: The system shall manage resources (medical kits, vehicles, personnel) associated with a mission.
- FR11: The system shall handle drone failures (loss of signal, low battery, hardware fault) with a defined recovery/emergency-landing procedure.
- FR12: The system shall generate a mission completion report summarizing the incident, zones covered, resources used, and outcomes.

1.2 Non-Functional Requirements

- Real-time communication: telemetry, alerts, and video must reach the command centre with minimal latency.
- Reliability & fault tolerance: the system must detect and gracefully handle drone/communication failures without losing mission data.
- Security: role-based authentication and encrypted communication channels for command and telemetry links.

- Scalability: the architecture must support many concurrent missions, drones, and zones during a large-scale disaster.
- Maintainability & extensibility: new sensor types, drone models, or notification channels should be addable with minimal impact (supported by low coupling / high cohesion design).
- Usability: dashboards for coordinators and operators must present mission-critical information clearly under stress.

1.3 System Boundary

The system boundary encloses the Command Dashboard, Mission/Incident/Alert management services, the telemetry link to drones, and the central data store. Physical rescue action in the field, the drone's flight hardware itself, and third-party weather/GIS data are treated as external to the software boundary but interact with it through well-defined interfaces, as shown in the Use Case Diagram (Section 4) and Deployment Diagram (Section 12).

2. Actor Identification

Actors were identified by analysing who or what initiates, or is affected by, interactions with the system boundary described in the scenario.

Actor	Type	Description
Emergency Coordinator	Primary (Human)	Registers incidents, defines zones, assigns drones/missions, coordinates teams, reviews reports.
Drone Operator	Primary (Human)	Plans routes, monitors and controls drone flight, responds to drone faults.
Field Rescue Team Member	Primary (Human)	Receives alerts, executes on-ground rescue actions, updates rescue status.
Drone	Primary (Autonomous)	Autonomously navigates, captures imagery/sensor data, detects victims/hazards, reports status.
Central Command System	Supporting (System)	Central software boundary that stores data and coordinates all subsystems (system-under-design).
Weather / GIS Service	Secondary (External System)	Supplies terrain, weather, and hazard-map data used to extend route planning.

3. Use Case Descriptions

The following table describes the primary use cases shown in the Use Case Diagram (Section 4), including actors, preconditions, main flow, and postconditions for the most critical scenarios.

UC-1: Register Disaster Incident

Primary Actor: Emergency Coordinator

Precondition: Coordinator is authenticated.

Main Flow: 1) Coordinator opens 'New Incident'. 2) Enters type, severity, location. 3) System validates and creates DisasterIncident. 4) System assigns unique incident ID.

Postcondition: A new DisasterIncident exists and is visible on the dashboard.

UC-2: Assign Drone to Mission

Primary Actor: Emergency Coordinator

Precondition: Incident and at least one affected zone exist; at least one drone is idle.

Main Flow: 1) Coordinator selects zone. 2) System lists available drones/operators. 3) Coordinator assigns drone + operator. 4) System creates RescueMission and triggers route planning (includes UC-5).

Postcondition: RescueMission is created with status 'Assigned'.

UC-5: Plan Route

Primary Actor: Drone Operator / System

Precondition: Mission and zone coordinates are available.

Main Flow: 1) System/operator requests a route. 2) System optionally queries Weather/GIS service (extend) for hazards. 3) Route with waypoints is computed and stored.

Postcondition: A Route object is linked to the mission.

UC-9: Detect Victim / Hazard

Primary Actor: Drone

Precondition: Drone is in 'Scanning' state within an assigned zone.

Main Flow: 1) Drone continuously captures imagery/sensor readings. 2) Onboard/edge analysis flags anomalies. 3) On a positive detection, the system includes UC-10 (Generate Alert).

Postcondition: Victim/hazard record created and linked to the zone.

UC-10: Generate Emergency Alert

Primary Actor: Central Command System

Precondition: A victim or hazard has been detected, or a drone fault has occurred.

Main Flow: 1) System creates an Alert with type and priority. 2) System notifies the relevant Field Rescue Team(s) (UC-11).

Postcondition: Alert is logged and at least one team is notified.

UC-13: Handle Drone Failure

Primary Actor: Drone / Drone Operator

Precondition: Drone reports a fault (signal loss, critical battery, hardware error).

Main Flow: 1) Drone/telemetry service detects the fault condition. 2) System switches drone to 'Fault / Emergency Landing' state. 3) Operator is notified and may take manual control. 4) System logs the event for the mission report.

Postcondition: Drone is safely landed/recovered or marked out-of-service; mission status updated.

UC-15: Generate Mission Completion Report

Primary Actor: Emergency Coordinator

Precondition: Mission status is 'Completed'.

Main Flow: 1) Coordinator requests report. 2) System compiles incident, zone, resource, and outcome data (includes UC-12, Manage Resources). 3) Report is generated and stored.

Postcondition: A MissionReport is available for review/export.

4. Use Case Diagram

The diagram below models the system boundary of ADDRRS together with its primary actors and use cases, including <<include>> relationships (e.g., every workflow includes authentication; victim detection includes alert generation) and <<extend>> relationships (e.g., route planning is extended by an optional weather/GIS check; drone monitoring is extended by fault handling when a fault condition arises).

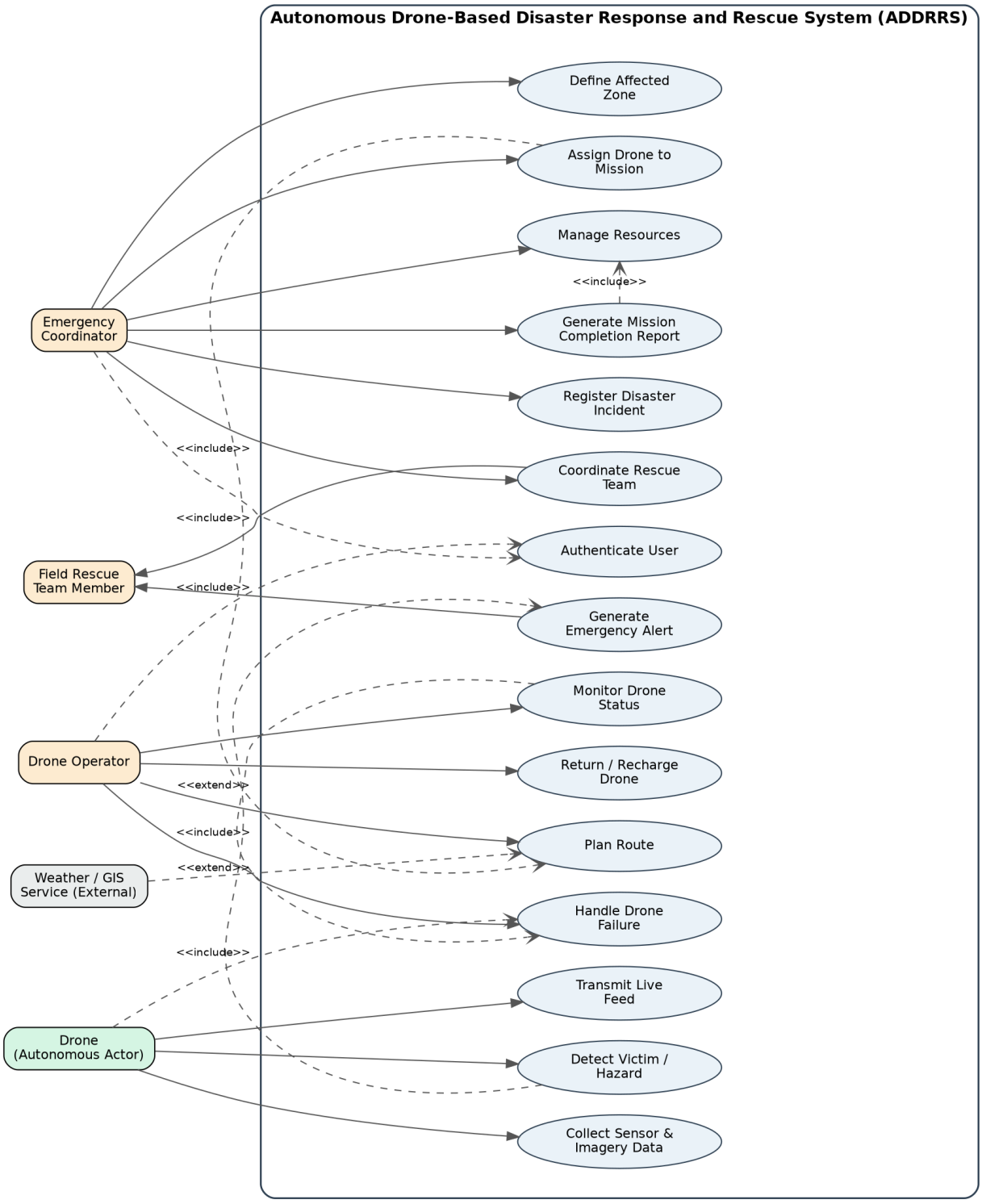

Figure 4.1: Use Case Diagram - ADDRRS

5. Object and Class Identification

Candidate classes were identified using noun-phrase analysis of the problem statement, then filtered against CRC (Class-Responsibility-Collaborator) criteria to remove duplicates, attributes mistaken for classes, and out-of-scope concepts.

Class	Category	Key Responsibility
User (abstract)	Actor/Entity	Common identity & authentication behaviour for all human roles.

EmergencyCoordinator	Entity	Registers incidents/zones, assigns missions, oversees resources & reports.
DroneOperator	Entity	Plans routes and operates/monitors assigned drones.
FieldTeamMember	Entity	Receives alerts and reports on-ground rescue status.
DisasterIncident	Entity	Represents a registered disaster event and groups its zones.
AffectedZone	Entity	Represents a geographic area needing survey/rescue, with a risk level.
RescueMission	Control/Entity	Coordinates the assignment of drone + route + resources to a zone; tracks mission status.
Drone	Entity (Autonomous)	Navigates, owns sensors/camera, reports telemetry, executes flight commands.
Camera	Entity	Captures images/video from the drone.
Sensor	Entity	Captures environmental readings (thermal, gas, etc.).
SensorData	Entity	Immutable record of a single sensor/camera capture with timestamp.
Route	Entity	Ordered waypoints computed for a mission.
Victim	Entity	Represents a detected person requiring rescue, with location & condition.
Alert	Control/Entity	Represents a generated emergency notification with type/priority.
Resource	Entity	Represents allocatable assets (medical kits, vehicles, personnel count).
CommandCenter	Control (Boundary)	Central coordinating object; monitors drones, dispatches missions.
MissionReport	Entity	Summarizes a completed mission for review/export.

5.1 Object Identification Approach

Object identification followed Coad & Yourdon's classification categories: (1) physical/tangible things - Drone, Camera, Sensor; (2) roles - EmergencyCoordinator, DroneOperator, FieldTeamMember; (3) events/transactions - RescueMission, Alert, SensorData capture; (4) interactions/associations - Route (links mission to a zone); and (5) places - AffectedZone. Attributes that only describe another class (e.g., battery level, GPS coordinates) were kept as attributes rather than promoted to classes, keeping the model at an appropriate level of granularity.

6. Class Diagram

The Class Diagram formalizes the classes from Section 5 together with their attributes, methods, multiplicities, and relationship types (generalization, composition, aggregation, association, and dependency).

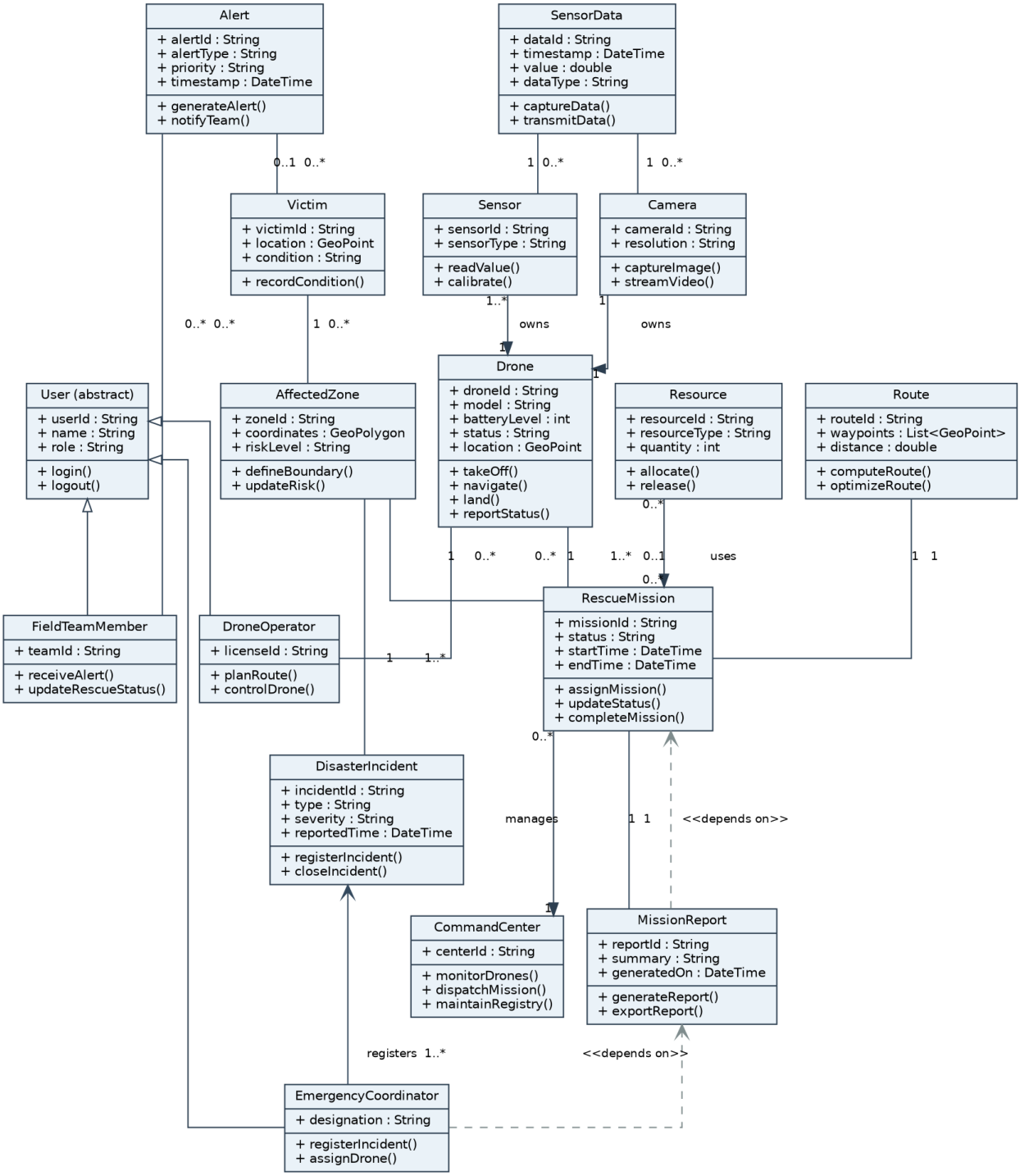

Figure 6.1: Class Diagram - ADDRRS

6.1 Relationship Justification

Relationship	Type	Justification
EmergencyCoordinator / DroneOperator / FieldTeamMember -> User	Generalization	All three roles share identity/authentication behaviour; modelled once in the superclass to avoid duplication.
Drone -> Camera, Drone -> Sensor	Composition	A Camera/Sensor is physically part of a specific Drone and has no independent lifecycle if the drone is decommissioned.
RescueMission -> Resource	Aggregation	Resources (vehicles, kits, personnel) can exist and be reused independently of any one mission.

RescueMission -> Drone, RescueMission -> Route	Association	A mission is temporarily linked to a drone and a computed route; the drone and route can exist without that specific mission afterward.
MissionReport -> RescueMission	Dependency	A report is generated from mission data at a point in time but does not permanently hold a structural reference.

7. Sequence / Interaction Diagram

The sequence diagram below traces one full scenario: a coordinator registers an incident and zone, a drone is assigned and scans the zone, a victim is detected, an alert is raised to a field team, and the mission is closed out with a report - covering the include relationships from the Use Case Diagram end-to-end.

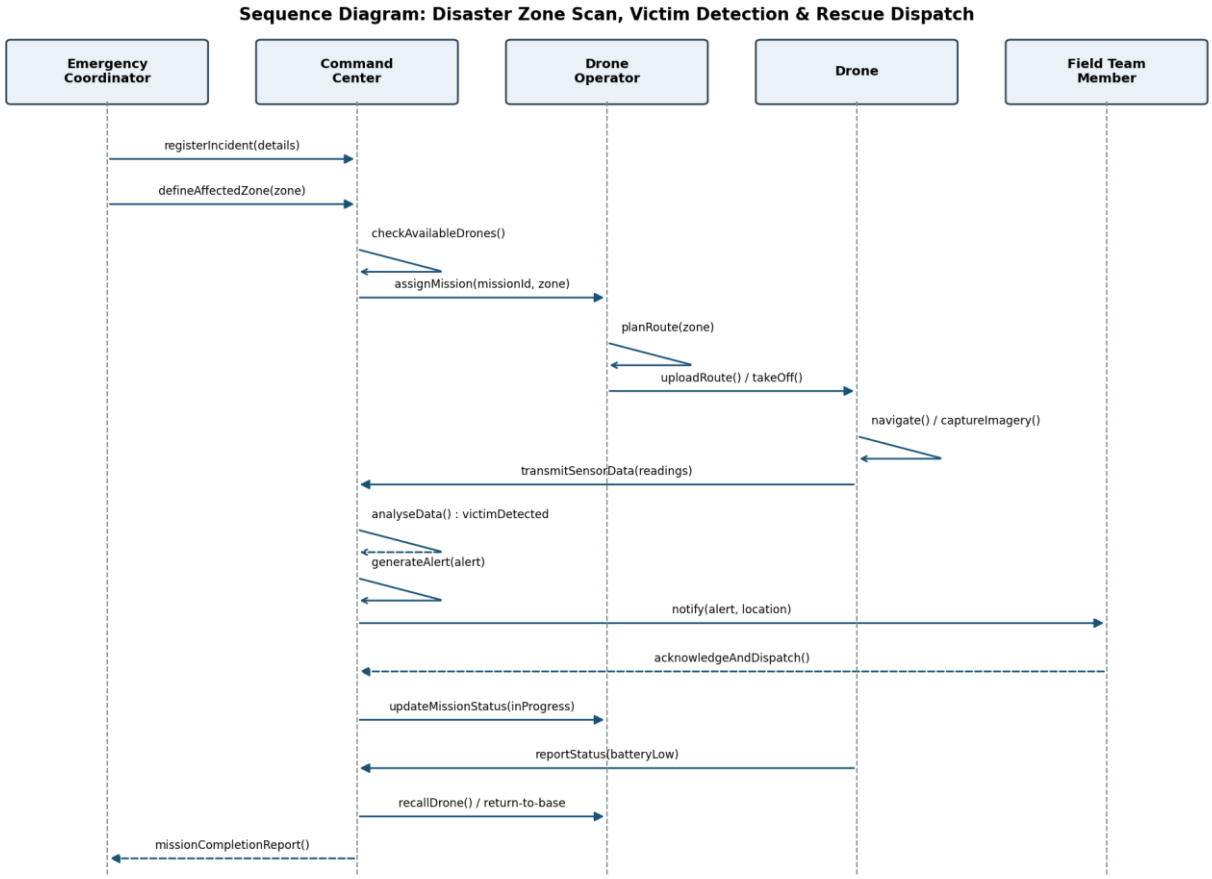

Figure 7.1: Sequence Diagram - Zone Scan, Victim Detection & Rescue Dispatch

8. Activity Diagram

The activity diagram models the end-to-end disaster-response workflow, including the decision points for drone availability, victim/hazard detection, and mission completion/battery status that drive the corresponding state transitions in Section 9.

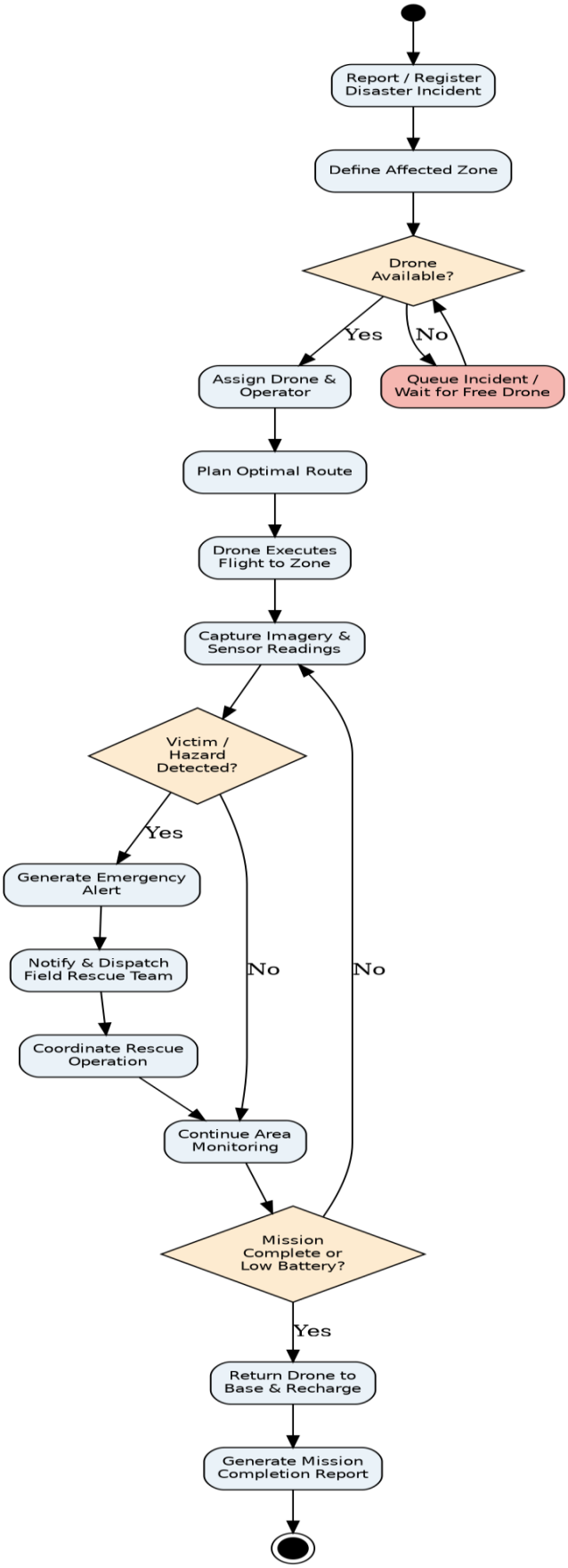

Figure 8.1: Activity Diagram - Disaster Response Workflow

9. State Transition Diagram

The Drone class exhibits the richest lifecycle in the system and is therefore chosen for the state transition diagram. States progress from Idle through pre-flight checks, en-route travel, scanning, optional victim-assistance mode, and return-to-base/charging, with a dedicated Fault state reachable from any active flight state.

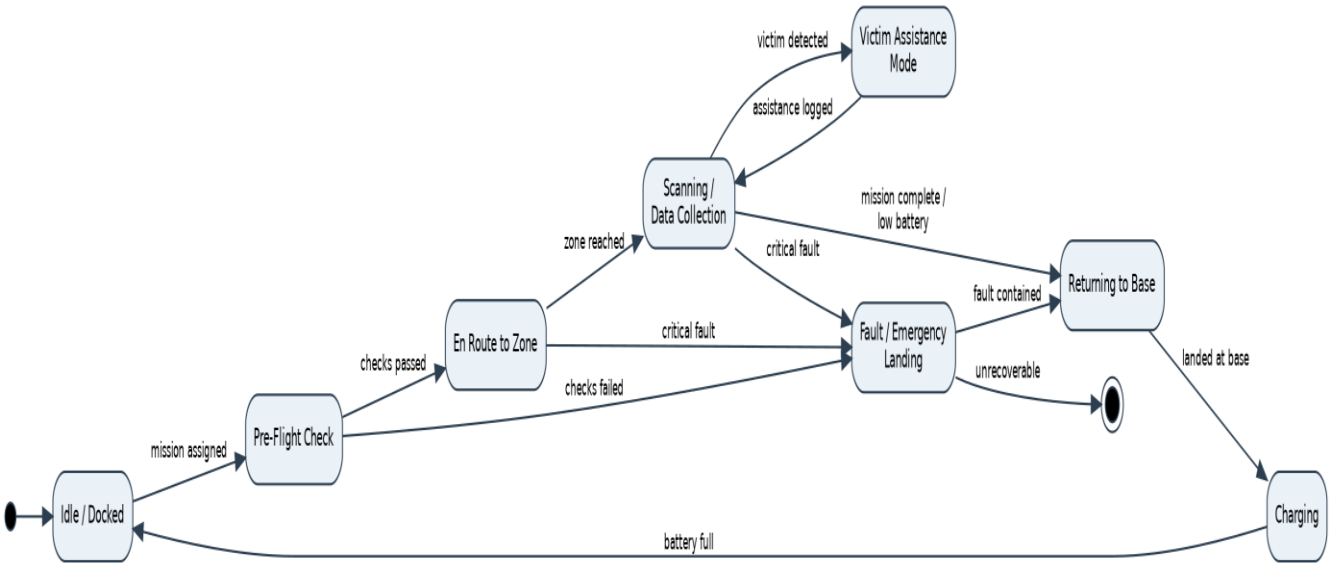

Figure 9.1: State Transition Diagram - Drone Lifecycle

10. Package Diagram

The system is decomposed into ten cohesive packages, separating presentation, domain services, and persistence. Dependencies point from higher-level workflow packages toward lower-level, more stable packages (e.g., Persistence), which keeps coupling directional and acyclic.

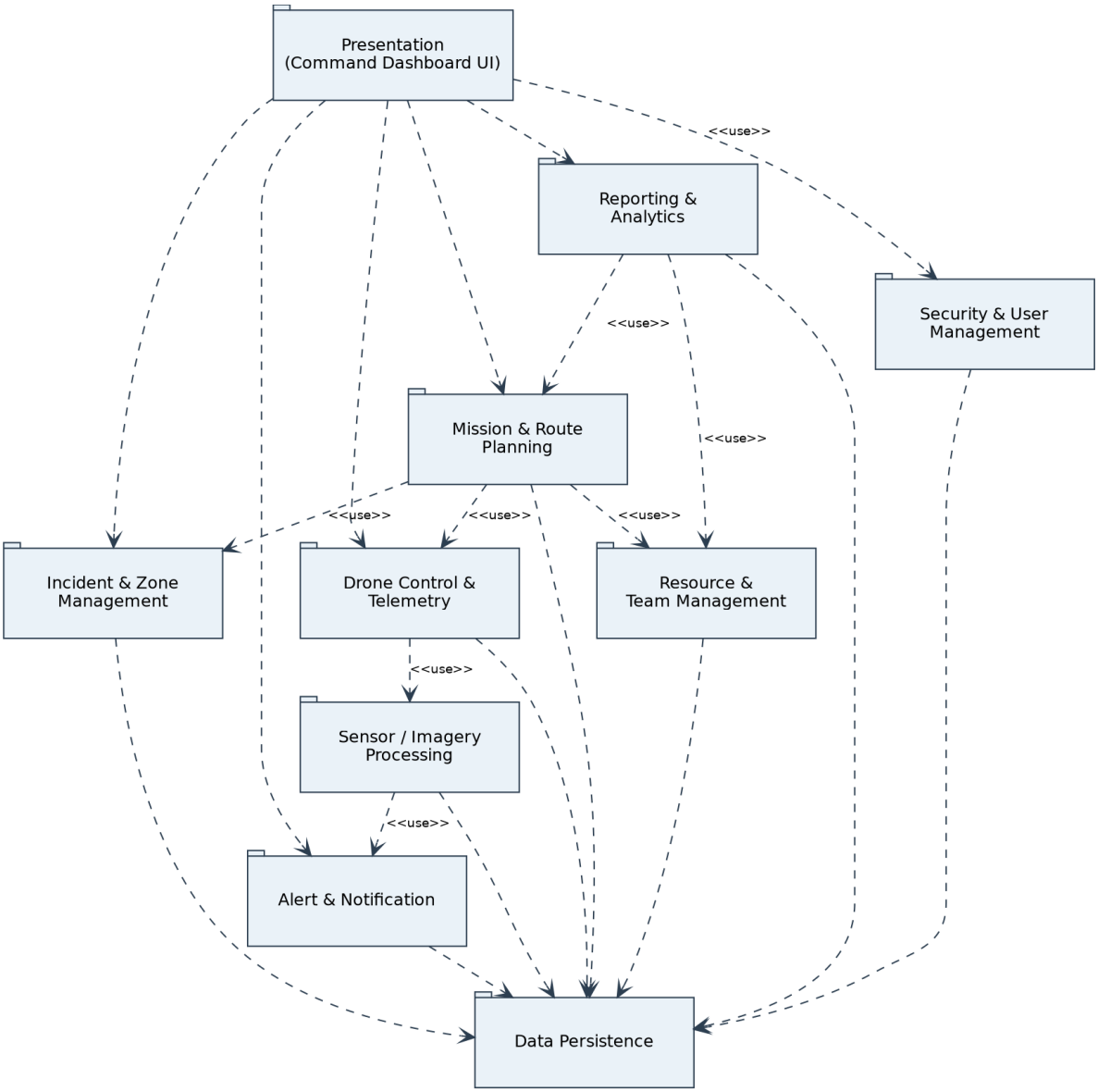

Figure 10.1: Package Diagram - ADDRRS Modules

11. Component Diagram

The component diagram refines the package view into deployable software components and their provided/required interfaces, showing the Command Dashboard as the sole client-facing component and the Telemetry & Communication Service as the single integration point with the drone's onboard component.

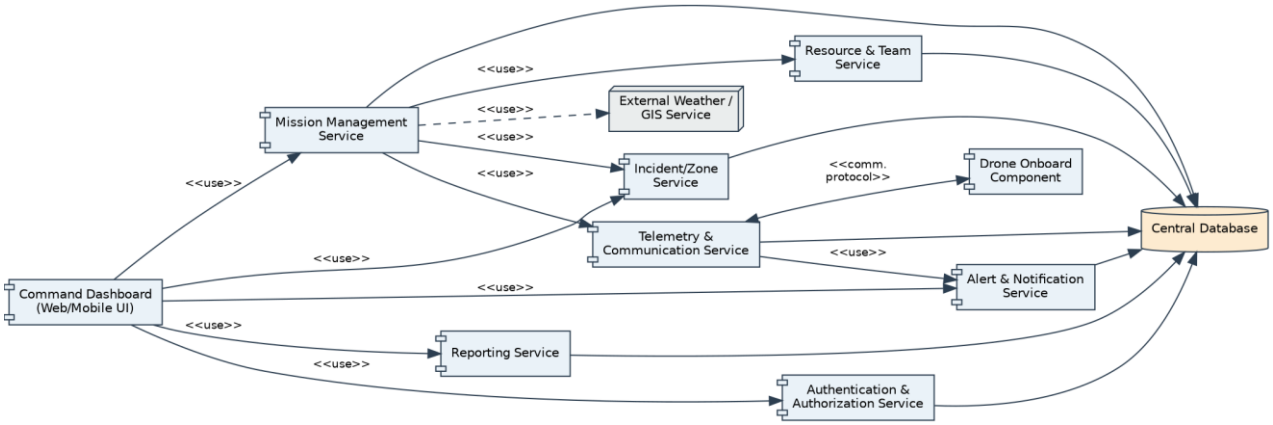

Figure 11.1: Component Diagram - ADDRRS

12. Deployment Diagram

Four physical/logical nodes are identified: the embedded Drone node (flight controller, edge-AI victim detection, sensor hardware), the Ground Control Station used by the operator, the Command Cloud/Data Center hosting application and database servers, and the Command Center Clients (coordinator web dashboard and field team mobile app). The external Weather/GIS provider is accessed over HTTPS from the application server.

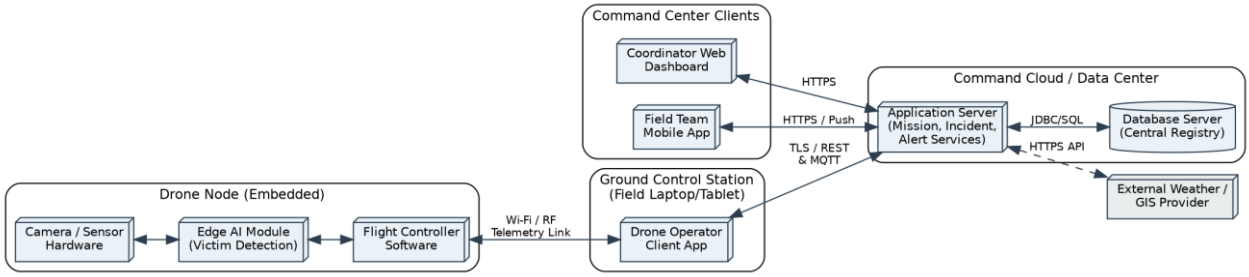

Figure 12.1: Deployment Diagram - ADDRRS

13. Object Responsibility Analysis

Responsibilities were assigned using CRC-style analysis so each class has a clear, singular purpose (supporting high cohesion) and collaborates with the minimum necessary set of other classes (supporting low coupling).

Class	Primary Responsibility	Key Collaborators
CommandCenter	Own the master registry of incidents, zones, missions and drones; dispatch missions; aggregate telemetry.	DisasterIncident, RescueMission, Drone
RescueMission	Bind a drone, route, zone, and resources together for the lifetime of one rescue effort; track status.	Drone, Route, AffectedZone, Resource
Drone	Execute flight commands, own its Camera/Sensors, report telemetry and detected anomalies.	Camera, Sensor, SensorData, CommandCenter
Alert	Encapsulate a single emergency notification and know how to notify the correct recipients.	FieldTeamMember, Victim
MissionReport	Summarize a completed mission for accountability and continuous improvement.	RescueMission, Resource

Assigning route computation to a dedicated Route class (rather than to Drone or RescueMission directly) follows the Expert pattern - Route has the waypoint data needed to justify owning computeRoute()/optimizeRoute() - and keeps Drone focused purely on flight execution.

14. Design Axioms, Principles and Design Pattern Justification

14.1 Design Axioms (Suh's Axiomatic Design)

Axiom 1 (Independence Axiom): each functional requirement is satisfied by an independent design parameter/class wherever possible - e.g., route planning (Route), telemetry (Drone/SensorData), and alerting (Alert) are separate classes rather than one large 'MissionManager' God class, so a change to alerting logic does not ripple into route computation.

Axiom 2 (Information Axiom): among independent designs, the simplest (lowest information content) is preferred - e.g., Drone state is modelled with a single 'status' attribute plus a well-defined state machine (Section 9) rather than multiple boolean flags that could produce inconsistent combinations.

14.2 Core Design Principles Applied

High Cohesion: Each class has one clear purpose (e.g., Alert only represents/dispatches a notification; it does not also manage resources).

Low Coupling: Classes collaborate through a small number of well-defined associations (e.g., RescueMission depends on Route and Drone but not on UI classes).

Encapsulation: Internal state (battery level, sensor calibration) is only modified through methods (e.g., reportStatus(), calibrate()), not exposed directly.

Abstraction: The abstract User class hides common authentication behaviour behind a simple interface used identically by all three role subclasses.

Single Responsibility Principle (SRP): MissionReport only formats/exports data; it does not also change mission state.

Open/Closed Principle (OCP): New sensor types can be added by extending Sensor without modifying Drone or SensorData.

Liskov Substitution: Any User subclass (Coordinator/Operator/FieldTeamMember) can be used wherever authentication behaviour is required.

14.3 Design Patterns Applied

Pattern	Problem Addressed	Application in ADDRRS
Observer	CommandCenter must react whenever a Drone reports new telemetry/alerts without polling.	Drone (subject) notifies CommandCenter/Dashboard (observers) on status change, decoupling drones from specific UI clients.
State	Drone behaviour differs sharply per lifecycle state (Idle, Scanning, Fault, etc.).	Encapsulates state-specific behaviour/transitions (Section 9) instead of large conditional blocks in Drone.
Strategy	Route optimization may need different algorithms (shortest path vs. hazard-avoiding) depending on conditions.	optimizeRoute() delegates to an interchangeable routing strategy, satisfying OCP.
Factory Method	Different Alert subtypes (victim-detected, drone-fault, hazard) must be created consistently.	An AlertFactory centralizes creation logic so RescueMission/Drone do not need to know alert construction details.

Facade	The Command Dashboard needs a simple entry point to many services (Incident, Mission, Alert, Reporting).	A CommandCenter/Facade component (Section 11) exposes a simplified interface, reducing UI coupling to internal services.

15. Assumptions and Constraints

15.1 Assumptions

- Drones are equipped with GPS, at least one camera, and at least one environmental sensor, and can maintain an intermittent telemetry link to the ground control station.
- Emergency personnel have network connectivity to the Command Dashboard during operations; brief link loss is tolerated and does not crash the drone's autonomous behaviour.
- Victim/hazard detection uses an onboard or edge-AI analysis step; the accuracy of that model is outside the scope of this OOAD exercise and is treated as a black-box collaborator.
- A single Central Database is sufficient for the scope of this design; distributed/replicated persistence is a future scaling concern, not part of the current class design.
- Field Rescue Team members carry a mobile device capable of receiving push notifications.

15.2 Constraints

- The design must clearly separate the system boundary, human actors, and autonomous components (Drone) as required by the problem statement.
- All UML diagrams must remain mutually consistent with the identified use cases (see Section 16 traceability).
- Real-time communication constraints limit how much processing can be pushed to the drone versus the command centre; time-critical detection logic is kept on the edge (Drone/EdgeAI), while heavier analytics remain server-side.
- Security constraints require authenticated, role-based access and encrypted telemetry, which shaped the inclusion of an explicit Authenticate User use case and Security & User Management package.
- Reliability constraints require an explicit Fault/Emergency-Landing state rather than allowing undefined behaviour on failure (Section 9).

16. Requirement-to-Use Case / Class Traceability

The table below demonstrates that every functional requirement from Section 1.1 is realized by at least one use case and implemented by at least one class, ensuring the model is complete and consistent.

Requirement	Use Case(s)	Realizing Class(es)
FR1	UC-1 Register Disaster Incident	DisasterIncident, EmergencyCoordinator
FR2	UC-1 (zone definition step)	AffectedZone
FR3	Authenticate User (included by all UCs)	User, EmergencyCoordinator, DroneOperator, FieldTeamMember
FR4	UC-2 Assign Drone to Mission, UC-5 Plan Route	RescueMission, Drone, Route
FR5	UC-6 Monitor Drone Status, UC-7 Collect Sensor Data	Drone, Camera, Sensor, SensorData
FR6	UC-8 Transmit Live Feed	Drone, SensorData, CommandCenter
FR7	UC-9 Detect Victim/Hazard, UC-10 Generate Alert	Victim, Alert, SensorData
FR8	UC-11 Coordinate Rescue Team	Alert, FieldTeamMember
FR9	UC-6 Monitor Drone Status	Drone, CommandCenter
FR10	UC-12 Manage Resources	Resource, RescueMission

FR11	UC-13 Handle Drone Failure	Drone (Fault state), DroneOperator
FR12	UC-15 Generate Mission Completion Report	MissionReport, RescueMission

17. Implementation / Prototype and Results

To validate that the identified classes and responsibilities are implementable and behave as intended, a lightweight Python console prototype was built covering the core scenario traced in the Sequence Diagram (Section 7): registering an incident, defining zones, assigning a drone to a mission, scanning zones, detecting a possible victim via sensor thresholds, raising an alert, and generating a mission report. The full source (addrrs_prototype.py) is included in the project's GitHub repository (Section 19).

17.1 Key Class Excerpt

```
class RescueMission:
    def __init__(self, mission_id, incident, drone):
        self.mission_id = mission_id
        self.incident = incident
        self.drone = drone
        self.status = "Assigned"

    def execute(self):
        self.status = "In Progress"
        self.drone.take_off()
        for zone in self.incident.zones:
            readings = self.drone.scan_zone(zone)
            if readings.get("Thermal", 0) > 38:
                alert = Alert("Possible Victim Detected", "HIGH")
                alert.notify_team("Field Rescue Team Alpha")
        self.drone.return_to_base()
        self.status = "Completed"
```

17.2 Sample Execution Output

```
[Incident INC-101] zone Z1 defined (13.0827N,80.2707E), risk=HIGH
[Incident INC-101] zone Z2 defined (13.0900N,80.2750E), risk=MODERATE

--- Mission M-2026-01 started (incident INC-101) ---
[Drone DR-07] taking off, status -> En Route
[Drone DR-07] scanning Z1: {'Thermal': 23.4, 'Gas': 41.2}, battery=85%
[Drone DR-07] scanning Z2: {'Thermal': 39.1, 'Gas': 26.4}, battery=70%
[Alert AL-001] 'Possible Victim Detected' (priority=HIGH) sent to Field Rescue Team
Alpha
[Drone DR-07] returning to base, battery=70%
--- Mission M-2026-01 status -> Completed ---

Mission Report [M-2026-01]
  Incident   : INC-101 (Flood, severity=High)
  Drone Used : DR-07 (Quad-X200)
  Zones      : ['Z1', 'Z2']
  Final Battery: 70%
  Status     : Completed
```

The trace confirms that: (a) zone definition correctly nests under an incident, (b) drone battery is consumed per scan as modelled by the 'battery low' transition in Section 9, (c) an Alert is only raised when

the sensor threshold condition of UC-9 is met, and (d) the final MissionReport aggregates exactly the fields traced in Section 16 (FR12).

18. Reflection on Design Decisions and Learning Outcomes

Working through this assignment reinforced that Object-Oriented Analysis and Design is most valuable at the boundary between human-operated and autonomous parts of a system: deciding whether the Drone should be modelled as an actor, an entity class, or both (it is both - an actor in the Use Case Diagram and a rich, stateful class in the Class/State diagrams) shaped almost every later diagram. Choosing composition for Drone-Camera/Sensor versus aggregation for RescueMission-Resource also clarified how lifecycle dependency, not just 'has-a' phrasing, should drive relationship selection.

Applying design axioms and patterns (Observer for telemetry, State for drone lifecycle, Strategy for route optimization) showed how the same functional requirement can be met with very different coupling/cohesion outcomes, and why the axiomatic 'independence' criterion is a practical checklist rather than an abstract rule. Building the small working prototype in Section 17 was particularly useful: implementing RescueMission.execute() surfaced that the alert-generation condition belongs logically closer to SensorData/Drone than to RescueMission, prompting a refinement in how responsibilities were assigned in Section 13.

In terms of Sustainable Development Goals, this system directly supports SDG 3 (Good Health and Well-being) and SDG 11 (Sustainable Cities and Communities) by reducing the time to locate and assist disaster victims; SDG 13 (Climate Action) is relevant because floods, storms, and other climate-linked disasters are a primary use case; and SDG 9 (Industry, Innovation and Infrastructure) is reflected in the use of autonomous, sensor-driven infrastructure to modernize emergency response. Overall, the exercise demonstrated that rigorous OOAD is not just documentation overhead - the traceability matrix in Section 16 and the working prototype in Section 17 both depended directly on decisions made during actor and class identification in Sections 2 and 5.

19. GitHub Upload

The complete assignment artefacts - this report, all UML diagram source files, and the working prototype - are organized for upload to a public or classroom GitHub repository as follows:

```
ADDRRS-OOAD-Assignment/
├── README.md                    (project overview, how to run the prototype)
├── docs/
│   ├── ADDRRS_Assignment.docx
│   ├── ADDRRS_Assignment.pdf
│   └── diagrams/                (usecase, class, sequence, activity,
│                                 state, package, component, deployment .png)
└── src/
    └── addrrs_prototype.py      (working console prototype)
```

GitHub Repository Link: _____

(Insert the HTTPS link to your repository here after uploading, and also paste the same link into the Google Classroom submission comment box.)